DevOps Private Kubernetes Project — Complete Step-by-Step Rebuild Guide

Every phase, every important command, configuration example, verification step, and the reason behind it.

Reference lab: Ubuntu 24.04.2 LTS • devops-lab • 172.16.4.110 • K3s v1.36.2+k3s1

Stack: Git/GitHub → GitHub Actions → Docker → Docker Hub → K3s/Kubernetes → Kustomize → Traefik → Ansible → Terraform/AWS S3

Design principle: Build one immutable Docker image using the Git commit SHA, then promote that same image through QA → UAT → Production.

Security: Replace all placeholders. Never commit secrets, tokens, private keys, AWS credentials, runner registration tokens, or Terraform state.

👤 Author

Shivam Yadav
DevOps Engineer

- LinkedIn: linkedin.com/in/shivamyd
- GitHub: github.com/Shivam-yd

Table of Contents

1. Architecture and Project Goal

The goal is to build a realistic private DevOps platform for a small Flask application. The Kubernetes server is private, so the GitHub self-hosted runner lives on the same server and connects outbound to GitHub.

Why each tool exists:

Tool	Purpose
Git/GitHub	Stores and versions source code
GitHub Actions	Automates CI/CD
Docker	Packages the app into an immutable artifact
Docker Hub	Stores the artifact
K3s/Kubernetes	Runs and manages containers
Kustomize	Creates environment-specific manifests from one base
Traefik	Routes HTTP requests by hostname
Ansible	Configures servers and runner prerequisites
Terraform	Manages infrastructure/state, not the Flask deployment
S3	Stores Terraform remote state

Important separation: Terraform provisions infrastructure; Ansible configures servers; Kubernetes deploys the application.

2. Prerequisites and Initial Server Checks

Step 1 — Check OS

Why: We need a known operating system so package and service commands behave predictably.

```
cat /etc/os-release
uname -a
```

Expected: Ubuntu 24.04.2 LTS in the reference lab.

Step 2 — Check hostname and network

Why: The runner and Kubernetes documentation use the server's identity and address.

```
hostname
hostname -I
ip addr
```

Expected: Hostname `devops-lab` and private IP `172.16.4.110`.

Step 3 — Check K3s

Why: K3s is the Kubernetes distribution used by the project.

```
systemctl status k3s
kubectl version --client
kubectl get nodes -o wide
kubectl cluster-info
```

Expected: The node is Ready and kubectl can communicate with the cluster.

Step 4 — Inspect existing workloads

Why: The lab server already has unrelated workloads; this prevents accidental deletion.

```
kubectl get namespaces
kubectl get pods -A
kubectl get svc -A
kubectl get ingress -A
```

Expected: Existing workloads are visible. Do not delete them.

Step 5 — Check required tools

Why: Missing tools should be fixed before starting the project.

```
git --version
docker --version
kubectl version --client
ansible --version
terraform version
aws --version
```

Expected: Each command returns a version.

Step 6 — Create the repository directory

Why: A consistent root directory makes all later relative paths predictable.

```
mkdir -p ~/devops-private-k8s-project
cd ~/devops-private-k8s-project
git init
```

Expected: A Git repository exists locally.

3. Phase 1 — Flask Application

Objective: Create a tiny application that is easy to understand but contains the two things we need for DevOps: configurable environment/version values and a health endpoint.

Step 1 — Create app directory

Why: The application code is separated from infrastructure files.

```
mkdir -p app
```

Step 2 — Create app.py

Why: The `/` endpoint demonstrates environment configuration; `/health` will later be used by Kubernetes probes and deployment verification.

```
from flask import Flask
import os

app = Flask(__name__)

@app.route("/")
def home():
    environment = os.getenv("ENVIRONMENT", "local")
    version = os.getenv("APP_VERSION", "1.0.0")
    return f"""
    <html>
    <head><title>DevOps Application</title></head>
    <body>
        <h1>Hello from DevOps Kubernetes Project!</h1>
        <p>Environment: {environment}</p>
        <p>Version: {version}</p>
    </body>
    </html>
    """

@app.route("/health")
def health():
    return "OK"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Step 3 — Create requirements.txt

Why: Pinning the Flask version makes dependency installation more repeatable.

```
Flask==3.1.2
```

Step 4 — Create a virtual environment

Why: It isolates project packages from system Python.

```
python3 -m venv venv
./venv/bin/pip install --upgrade pip
./venv/bin/pip install -r app/requirements.txt
```

Expected: Flask installs without errors.

Step 5 — Run Flask locally

Why: We must prove the application works before introducing Docker.

```
./venv/bin/python app/app.py
# In another terminal:
curl http://127.0.0.1:5000
curl http://127.0.0.1:5000/health
```

Expected: The page loads and `/health` returns `OK`.

Step 6 — Test configuration

Why: Environment variables prove that the same code can display different environment/version values.

```
ENVIRONMENT=test APP_VERSION=9.9.9 ./venv/bin/python app/app.py
# In another terminal:
curl http://127.0.0.1:5000
```

Expected: The response contains `Environment: test` and `Version: 9.9.9`.

4. Phase 2 — Docker and Docker Hub

Objective: Turn the Flask application into a portable image. The final image runs as a non-root user.

Step 1 — Create Dockerfile

Why: The Dockerfile defines the reproducible application image and its runtime user.

```
FROM python:3.12-slim
WORKDIR /app
RUN useradd --create-home --shell /usr/sbin/nologin appuser
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY --chown=appuser:appuser app.py .
EXPOSE 5000
ENV ENVIRONMENT=local
ENV APP_VERSION=1.0.0
USER appuser
CMD ["python", "app.py"]
```

Step 2 — Create .dockerignore

Why: It keeps local development files and possible secrets out of the build context.

```
venv/
__pycache__/
*.pyc
.git/
.gitignore
README.md
.env
*.env
```

Step 3 — Build image

Why: A local build catches Dockerfile errors before CI.

```
docker build -t devops-app:test ./app
docker images | grep devops-app
```

Expected: `devops-app:test` exists.

Step 4 — Run and test container

Why: This isolates container problems from Kubernetes problems.

```
docker run -d --name devops-test -p 5000:5000 devops-app:test
curl http://127.0.0.1:5000
curl http://127.0.0.1:5000/health
```

Expected: The page loads and health returns `OK`.

Step 5 — Verify non-root user

Why: A non-root application has fewer privileges if compromised.

```
docker exec devops-test whoami
```

Expected: `appuser`.

Step 6 — Clean up

Why: Abandoned containers waste resources.

```
docker rm -f devops-test
```

Step 7 — Push a versioned image

Why: Kubernetes needs an image registry from which it can pull.

```
docker login
docker tag devops-app:test YOUR_DOCKERHUB_USERNAME/devops-app:1.0.0
docker push YOUR_DOCKERHUB_USERNAME/devops-app:1.0.0
docker pull YOUR_DOCKERHUB_USERNAME/devops-app:1.0.0
```

Expected: The image can be pulled from Docker Hub.

> *Later CI uses `github.sha` instead of a mutable tag such as `latest`. That gives exact source-to-image traceability.*

5. Phase 3 — K3s/Kubernetes Manual Deployment

Objective: First understand the basic Kubernetes objects manually: Namespace → Deployment → Pods → Service. CI/CD is added only after this foundation works.

Step 1 — Verify K3s

Why: K3s is the cluster platform.

```
systemctl status k3s
kubectl get nodes -o wide
kubectl get pods -A
```

Expected: K3s is active and `devops-lab` is Ready.

Step 2 — Create QA namespace

Why: Namespaces logically isolate application environments.

```
kubectl create namespace qa
kubectl get namespaces
```

Expected: `qa` exists.

Step 3 — Create Deployment

Why: A Deployment maintains the desired number of Pods and manages replacement/rolling updates.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: devops-app
  namespace: qa
spec:
  replicas: 2
  selector:
    matchLabels:
      app: devops-app
  template:
    metadata:
      labels:
        app: devops-app
    spec:
      containers:
        - name: devops-app
          image: YOUR_DOCKERHUB_USERNAME/devops-app:1.0.0
          ports:
            - containerPort: 5000
          env:
            - name: ENVIRONMENT
              value: qa
            - name: APP_VERSION
              value: 1.0.0
          readinessProbe:
            httpGet:
              path: /health
              port: 5000
            initialDelaySeconds: 5
            periodSeconds: 10
          livenessProbe:
            httpGet:
              path: /health
              port: 5000
            initialDelaySeconds: 15
            periodSeconds: 20
```

```
kubectl apply -f /tmp/qa-deployment.yaml
```

Step 4 — Create Service

Why: Pods are replaceable; the Service gives them a stable network endpoint.

```
apiVersion: v1
kind: Service
metadata:
  name: devops-app
  namespace: qa
spec:
  type: ClusterIP
  selector:
    app: devops-app
  ports:
    - protocol: TCP
      port: 5000
      targetPort: 5000
```

```
kubectl apply -f /tmp/qa-service.yaml
```

Step 5 — Verify resources

Why: We confirm the desired state became a working state.

```
kubectl get pods -n qa
kubectl get deployment -n qa
kubectl get service -n qa
kubectl describe deployment devops-app -n qa
```

Expected: Two Pods are Running/Ready and the Service is ClusterIP.

Step 6 — Test using port-forward

Why: Port-forward lets us test the Service without exposing it externally.

```
kubectl port-forward -n qa svc/devops-app 5001:5000
# Another terminal:
curl http://127.0.0.1:5001
curl http://127.0.0.1:5001/health
```

Expected: `OK` from `/health`.

Step 7 — Test self-healing

Why: Kubernetes should correct a difference between desired and actual state.

```
kubectl get pods -n qa
kubectl delete pod <pod-name> -n qa
kubectl get pods -n qa -w
```

Expected: A replacement Pod appears.

Step 8 — Test scaling

Why: Changing replicas demonstrates horizontal capacity at the Deployment level.

```
kubectl scale deployment devops-app -n qa --replicas=3
kubectl get pods -n qa
kubectl scale deployment devops-app -n qa --replicas=2
```

Expected: The number of Pods changes to 3 and then back to 2.

> ***Readiness*** *— can this Pod receive traffic?* ***Liveness*** *— is this container still healthy? These probes become important during rolling updates and failures.*

6. Phase 3B — Kustomize Base and Environment Overlays

Objective: Create one reusable Kubernetes base and three small environment overlays instead of maintaining three complete copies of the same YAML.

```
kubernetes/
├── base/
│   ├── deployment.yaml
│   ├── service.yaml
│   └── kustomization.yaml
└── environments/
    ├── qa/
    │   ├── kustomization.yaml
    │   └── ingress.yaml
    ├── uat/
    │   ├── kustomization.yaml
    │   └── ingress.yaml
    └── production/
        ├── kustomization.yaml
        └── ingress.yaml
```

Step 1 — Create directories

Why: This creates the intended configuration hierarchy.

```
mkdir -p kubernetes/base
mkdir -p kubernetes/environments/qa
mkdir -p kubernetes/environments/uat
mkdir -p kubernetes/environments/production
```

Step 2 — Create base Deployment

Why: The base contains common application settings, probes, labels and resource controls.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: devops-app
  labels:
    app: devops-app
    project: devops-private-k8s-project
spec:
  replicas: 1
  selector:
    matchLabels:
      app: devops-app
  template:
    metadata:
      labels:
        app: devops-app
        project: devops-private-k8s-project
    spec:
      containers:
        - name: devops-app
          image: YOUR_DOCKERHUB_USERNAME/devops-app:1.0.0
          ports:
            - containerPort: 5000
          env:
            - name: ENVIRONMENT
              value: unknown
            - name: APP_VERSION
              value: 1.0.0
          resources:
            requests:
              cpu: 100m
              memory: 128Mi
            limits:
              cpu: 500m
              memory: 256Mi
          readinessProbe:
            httpGet:
              path: /health
              port: 5000
            initialDelaySeconds: 5
            periodSeconds: 10
          livenessProbe:
            httpGet:
              path: /health
              port: 5000
            initialDelaySeconds: 15
            periodSeconds: 20
```

Step 3 — Create base Service

Why: ClusterIP keeps application traffic internal and allows Traefik to be the external entry point.

```
apiVersion: v1
kind: Service
metadata:
  name: devops-app
spec:
  type: ClusterIP
  selector:
    app: devops-app
  ports:
    - protocol: TCP
      port: 5000
      targetPort: 5000
```

Step 4 — Create base kustomization

Why: It tells Kustomize which common files belong to the base.

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - deployment.yaml
  - service.yaml
```

Step 5 — Create QA overlay

Why: QA needs its own namespace, two replicas and environment value while reusing the base.

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: qa
resources:
  - ../../base
images:
  - name: YOUR_DOCKERHUB_USERNAME/devops-app
    newTag: IMAGE_TAG_PLACEHOLDER
replicas:
  - name: devops-app
    count: 2
patches:
  - target:
      kind: Deployment
      name: devops-app
    patch: |-
      - op: replace
        path: /spec/template/spec/containers/0/env/0/value
        value: qa
```

Step 6 — Create UAT overlay

Why: UAT uses the same base but its own environment settings.

Copy the QA overlay and change: - namespace: `uat` - replicas count: `2` - environment value: `uat`

Step 7 — Create Production overlay

Why: Production needs the same application but a larger replica count and production configuration.

Copy the QA overlay and change: - namespace: `production` - replicas count: `3` - environment value: `production`

Step 8 — Render manifests

Why: Rendering lets us inspect the final YAML before it reaches Kubernetes.

```
kubectl kustomize kubernetes/environments/qa
kubectl kustomize kubernetes/environments/uat
kubectl kustomize kubernetes/environments/production
```

Expected: Correct namespaces, replicas and environment values appear.

Step 9 — Apply all environments

Why: This creates the desired QA/UAT/Production resources.

```
kubectl apply -k kubernetes/environments/qa
kubectl apply -k kubernetes/environments/uat
kubectl apply -k kubernetes/environments/production
```

Step 10 — Verify environment separation

Why: It proves that configuration differs without duplicating application code.

```
kubectl exec -n qa deployment/devops-app -- printenv ENVIRONMENT
kubectl exec -n uat deployment/devops-app -- printenv ENVIRONMENT
kubectl exec -n production deployment/devops-app -- printenv ENVIRONMENT
```

Expected: `qa`, `uat`, `production`.

7. Phase 4 — Ansible Server Automation

Objective: Automate server configuration. Ansible is used here for packages, checks, the runner user and runner directory — not for provisioning AWS infrastructure.

```
ansible/
├── ansible.cfg
├── inventory/hosts
├── group_vars/kubernetes.yml
├── playbook.yml
└── roles/
    ├── kubernetes/tasks/main.yml
    └── github_runner/tasks/main.yml
```

Step 1 — Create Ansible directories

Why: Roles separate server configuration into reusable components.

```
mkdir -p ansible/inventory ansible/group_vars
mkdir -p ansible/roles/kubernetes/tasks
mkdir -p ansible/roles/github_runner/tasks
mkdir -p ansible/roles/github_runner/templates
```

Step 2 — Create inventory

Why: Inventory identifies the server Ansible manages.

```
[kubernetes]
devops-lab ansible_host=172.16.4.110 ansible_user=root
```

Step 3 — Create ansible.cfg

Why: Central settings avoid repeating inventory and connection options.

```
[defaults]
inventory = inventory/hosts
host_key_checking = False
retry_files_enabled = False
interpreter_python = auto_silent

[ssh_connection]
pipelining = True
```

Step 4 — Create variables

Why: Variables make server values easy to change without editing tasks.

```
kubernetes_server_ip: "172.16.4.110"
kubernetes_server_hostname: "devops-lab"
k3s_service_name: "k3s"
project_directory: "/root/devops-private-k8s-project"
github_runner_user: "actions"
github_runner_directory: "/home/actions/actions-runner"
github_repo_owner: "YOUR_GITHUB_USERNAME"
github_repo_name: "devops-private-k8s-project"
```

Step 5 — Create playbook

Why: The playbook selects the roles that configure the target.

```
---
- name: Configure Kubernetes server
  hosts: kubernetes
  become: false
  roles:
    - kubernetes
    - github_runner
```

Step 6 — Create Kubernetes role

Why: The role installs prerequisites and verifies the existing K3s system.

```
---
- name: Install required packages
  apt:
    name:
      - curl
      - git
      - ca-certificates
      - unzip
    state: present
    update_cache: true

- name: Show hostname
  command: hostname
  register: hostname_output
  changed_when: false

- name: Check K3s
  command: systemctl is-active k3s
  register: k3s_status
  changed_when: false

- name: Check kubectl
  command: kubectl version --client
  changed_when: false

- name: Get Kubernetes nodes
  command: kubectl get nodes
  changed_when: false

- name: Check disk
  command: df -h
  changed_when: false
```

Step 7 — Create GitHub runner role

Why: The role creates a dedicated non-root user and working directory. The short-lived GitHub registration token is intentionally not stored in Ansible.

```
---
- name: Install runner dependencies
  apt:
    name:
      - curl
      - git
      - jq
      - unzip
      - ca-certificates
    state: present
    update_cache: true

- name: Create actions user
  user:
    name: "{{ github_runner_user }}"
    create_home: true
    shell: /bin/bash

- name: Create runner directory
  file:
    path: "{{ github_runner_directory }}"
    state: directory
    owner: "{{ github_runner_user }}"
    group: "{{ github_runner_user }}"
    mode: "0755"
```

Step 8 — Test connectivity

Why: A ping test catches SSH/inventory problems before a full playbook run.

```
cd ansible
ansible all -m ping
```

Expected: `pong`.

Step 9 — Run playbook

Why: Now server configuration is repeatable.

```
ansible-playbook playbook.yml
```

Expected: No fatal errors; `actions` user/directory exist.

> ***Production improvement:*** *use a dedicated administrative account with sudo and strict SSH host-key verification.* `root` + `host_key_checking=False` *is a lab simplification.*

8. Phase 4B — GitHub Self-Hosted Runner

Objective: Let GitHub Actions execute inside the private Kubernetes network. The runner makes an outbound HTTPS connection to GitHub, so the Kubernetes API does not need public inbound access.

Step 1 — Create runner from GitHub UI

Why: GitHub provides the current runner package URL and registration commands. These commands can change, so use the current command shown by GitHub.

Repository → Settings → Actions → Runners → New self-hosted runner → Linux x64

Expected: A current runner download command and temporary registration token.

Step 2 — Use the actions user

Why: The runner should not run workflow code as root.

```
su - actions
cd /home/actions/actions-runner
```

Step 3 — Download and extract

Why: The runner package installs the agent that communicates with GitHub.

```
# Run the exact current download command shown by GitHub.
tar xzf actions-runner.tar.gz
ls -la
```

Expected: Runner files are present.

Step 4 — Register

Why: Registration associates the machine with the repository.

```
./config.sh --url https://github.com/OWNER/REPOSITORY --token <TEMP_TOKEN>
```

Expected: Use runner name `devops-lab` and labels `self-hosted, linux, k3s`.

Step 5 — Install service

Why: A systemd service starts the runner automatically and survives reboot.

```
sudo ./svc.sh install actions
sudo ./svc.sh start
sudo ./svc.sh status
```

Expected: Runner service is active.

Step 6 — Verify GitHub runner

Why: The workflow can only target a runner that is online and has matching labels.

```
systemctl --type=service | grep actions.runner
journalctl -u <runner-service-name> -n 50 --no-pager
```

Expected: GitHub shows the runner Online.

> *Never store the runner registration token in Git, Ansible, README files or chat. A self-hosted runner executes workflow code directly on your infrastructure.*

9. Phase 5 — GitHub Actions CI

Objective: Automate source testing and Docker image creation. The Docker job must wait for the test job.

Step 1 — Create Docker Hub secrets

Why: Secrets keep registry credentials out of source code.

Repository → Settings → Secrets and variables → Actions

Create: `DOCKERHUB_USERNAME`, `DOCKERHUB_TOKEN`

Expected: Secrets exist and the token is an access token.

Step 2 — Create ci.yml

Why: The workflow implements test → build → push.

```
mkdir -p .github/workflows
```

```
name: CI

on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main

jobs:
  test:
    runs-on: [self-hosted, linux, k3s]
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Check Python
        run: python3 --version
      - name: Create virtual environment
        run: python3 -m venv venv
      - name: Install dependencies
        run: |
          ./venv/bin/pip install --upgrade pip
          ./venv/bin/pip install -r app/requirements.txt
      - name: Test application
        run: |
          ./venv/bin/python -m py_compile app/app.py

  docker:
    needs: test
    runs-on: [self-hosted, linux, k3s]
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Login to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}
      - name: Build Docker image
        run: |
          docker build -t ${{ secrets.DOCKERHUB_USERNAME }}/devops-app:${{ github.sha }} ./app
      - name: Push Docker image
        run: |
          docker push ${{ secrets.DOCKERHUB_USERNAME }}/devops-app:${{ github.sha }}
```

Step 3 — Commit and push

Why: A push to `main` starts the CI workflow.

```
git add .
git commit -m "Add CI pipeline"
git push origin main
```

Expected: Test passes, then Docker build/push runs.

Step 4 — Verify SHA image

Why: The registry should contain the exact artifact created from this commit.

```
docker pull YOUR_DOCKERHUB_USERNAME/devops-app:<GIT_SHA>
```

Expected: The image pulls successfully.

> ***Why github.sha?*** *If two builds are both called* `latest`*, you cannot reliably identify which code is running. A commit SHA provides immutable traceability.*

10. Phase 6 — Automatic QA Deployment

Objective: Deploy the exact CI artifact to QA automatically. No rebuild is performed during deployment.

Step 1 — Add the image placeholder

Why: Kustomize needs a value that CI can replace with the current commit SHA.

```
images:
  - name: YOUR_DOCKERHUB_USERNAME/devops-app
    newTag: IMAGE_TAG_PLACEHOLDER
```

Step 2 — Add deploy-qa job

Why: The `needs` dependency guarantees that Docker push completed first.

```
deploy-qa:
  needs: docker
  runs-on: [self-hosted, linux, k3s]
  steps:
    - name: Checkout code
      uses: actions/checkout@v4
    - name: Deploy to QA
      run: |
        kubectl kustomize kubernetes/environments/qa | sed "s|IMAGE_TAG_PLACEHOLDER|${{ github.sha }}|g" | kubectl apply -f -
    - name: Wait for deployment
      run: |
        kubectl rollout status deployment/devops-app -n qa --timeout=120s
    - name: Check QA pods
      run: kubectl get pods -n qa
    - name: Check QA service
      run: kubectl get service -n qa
```

Step 3 — Verify QA

Why: Rollout status proves Pods become ready, not merely that kubectl accepted YAML.

```
kubectl rollout status deployment/devops-app -n qa
kubectl get pods -n qa
kubectl get deployment devops-app -n qa -o jsonpath='{.spec.template.spec.containers[0].image}'
```

Expected: Pods are Ready and the image contains the current SHA.

Step 4 — Verify QA configuration

Why: The overlay must set the QA environment.

```
kubectl exec -n qa deployment/devops-app -- printenv ENVIRONMENT
```

Expected: `qa`.

11. Phase 7 — UAT Manual Approval

Objective: Create a controlled promotion gate between QA and UAT.

Step 1 — Create UAT GitHub Environment

Why: Environments provide deployment protection rules and auditability.

Repository → Settings → Environments → New environment: `uat` → Required reviewers

Expected: UAT requires a reviewer.

Step 2 — Add deploy-uat job

Why: The environment gate stops the job until approval.

```
deploy-uat:
  needs: deploy-qa
  runs-on: [self-hosted, linux, k3s]
  environment:
    name: uat
  steps:
    - uses: actions/checkout@v4
    - name: Deploy to UAT
      run: |
        kubectl kustomize kubernetes/environments/uat | sed "s|IMAGE_TAG_PLACEHOLDER|${{ github.sha }}|g" | kubectl apply -f -
    - name: Wait for UAT
      run: |
        kubectl rollout status deployment/devops-app -n uat --timeout=120s
    - name: Check UAT
      run: kubectl get pods -n uat
```

Step 3 — Approve UAT

Why: A human verifies the QA result before promotion.

Open the workflow run → review the pending `uat` deployment → approve.

Expected: The UAT job continues.

Step 4 — Verify UAT

Why: We verify health, environment and image identity.

```
kubectl get pods -n uat
kubectl exec -n uat deployment/devops-app -- printenv ENVIRONMENT
kubectl get deployment devops-app -n uat -o jsonpath='{.spec.template.spec.containers[0].image}'
```

Expected: `ENVIRONMENT=uat` and image SHA matches QA.

12. Phase 8 — Production Manual Approval

Objective: Add a second protection gate before Production. Production uses three replicas.

Step 1 — Create production Environment

Why: This creates an auditable Production approval boundary.

Repository → Settings → Environments → New environment: `production` → Required reviewers

Expected: Production requires approval.

Step 2 — Add production job

Why: The job depends on UAT and the production environment.

```
deploy-production:
  needs: deploy-uat
  runs-on: [self-hosted, linux, k3s]
  environment:
    name: production
  steps:
    - uses: actions/checkout@v4
    - name: Deploy to Production
      run: |
        kubectl kustomize kubernetes/environments/production | sed "s|IMAGE_TAG_PLACEHOLDER|${{ github.sha }}|g" | kubectl apply -f -
    - name: Wait for Production
      run: |
        kubectl rollout status deployment/devops-app -n production --timeout=120s
    - name: Check Production
      run: kubectl get pods -n production
```

Step 3 — Approve Production

Why: A reviewer makes the final release decision.

Open workflow → review UAT result → approve production.

Expected: Production deployment runs.

Step 4 — Verify Production

Why: Three replicas and production configuration must be healthy.

```
kubectl get pods -n production
kubectl get deployment devops-app -n production
kubectl exec -n production deployment/devops-app -- printenv ENVIRONMENT
kubectl get deployment devops-app -n production -o jsonpath='{.spec.template.spec.containers[0].image}'
```

Expected: 3 Pods Ready, `ENVIRONMENT=production`, same SHA as QA/UAT.

13. Phase 9 — Traefik Ingress

Objective: Provide user-friendly hostnames and keep application Services internal. K3s already includes Traefik, so verify it instead of installing another controller.

Step 1 — Verify Traefik

Why: We need to know the ingress controller and its exposure before testing routing.

```
kubectl get pods -n kube-system | grep -i traefik
kubectl get svc -n kube-system | grep -i traefik
kubectl get ingressclass
```

Expected: Traefik is running and the `traefik` IngressClass exists.

Step 2 — Create QA ingress

Why: Host-based routing sends `qa.devops.local` to the QA Service.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: devops-app
  namespace: qa
spec:
  ingressClassName: traefik
  rules:
    - host: qa.devops.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: devops-app
                port:
                  number: 5000
```

Step 3 — Create UAT/Production ingress

Why: Each environment gets a unique hostname and namespace.

- **UAT:** host `uat.devops.local`, namespace `uat`
- **Production:** host `prod.devops.local`, namespace `production`

Step 4 — Include ingress in overlays

Why: Kustomize must render the ingress together with the base resources.

```
resources:
  - ../../base
  - ingress.yaml
```

Step 5 — Check existing ingress first

Why: The server may have unrelated ingress resources; we must not overwrite/delete them.

```
kubectl get ingress -A
```

Expected: Existing resources remain intact.

Step 6 — Apply ingress

Why: This creates the routing rules.

```
kubectl apply -k kubernetes/environments/qa
kubectl apply -k kubernetes/environments/uat
kubectl apply -k kubernetes/environments/production
kubectl get ingress -A
```

Expected: Three application ingress resources are visible.

Step 7 — Configure Windows hosts file

Why: The `.local` names are lab-only and are not automatically resolved by public DNS.

Edit as Administrator: `C:\Windows\System32\drivers\etc\hosts`

```
172.16.4.110 qa.devops.local
172.16.4.110 uat.devops.local
172.16.4.110 prod.devops.local
```

Step 8 — Test routing

Why: Host headers prove Traefik routes to the correct namespace.

```
curl -H "Host: qa.devops.local" http://172.16.4.110/health
curl -H "Host: uat.devops.local" http://172.16.4.110/health
curl -H "Host: prod.devops.local" http://172.16.4.110/health
```

Expected: Each returns `OK`.

14. Phase 10 — End-to-End Testing and Rollback

Objective: Prove the complete release chain and prove recovery from failures.

Step 1 — Make a visible application change

Why: A version/heading change provides evidence that a new commit moved through the pipeline.

```
# Edit app/app.py
# Example: change the page heading or default APP_VERSION.
git add .
git commit -m "Release version 2.0.0"
git push origin main
```

Expected: A new GitHub Actions run begins.

Step 2 — Follow the pipeline

Why: The dependency chain demonstrates controlled promotion.

```
test -> docker -> deploy-qa -> UAT approval -> deploy-uat -> production approval -> deploy-production
```

Expected: Each stage waits for its prerequisite.

Step 3 — Compare image SHA

Why: The same immutable artifact must be promoted.

```
kubectl get deployment devops-app -n qa -o jsonpath='{.spec.template.spec.containers[0].image}'; echo
kubectl get deployment devops-app -n uat -o jsonpath='{.spec.template.spec.containers[0].image}'; echo
kubectl get deployment devops-app -n production -o jsonpath='{.spec.template.spec.containers[0].image}'; echo
```

Expected: All three show the same Git SHA.

Step 4 — Test user-facing health

Why: This validates the complete route through Traefik.

```
curl http://qa.devops.local/health
curl http://uat.devops.local/health
curl http://prod.devops.local/health
```

Expected: `OK` from all three.

Step 5 — Test rolling update

Why: Kubernetes can replace Pods gradually instead of stopping all replicas at once.

```
kubectl rollout history deployment/devops-app -n production
kubectl rollout status deployment/devops-app -n production
```

Expected: Rollout completes.

Step 6 — Test rollback

Why: Rollback is a fast operational recovery mechanism.

```
kubectl rollout history deployment/devops-app -n production
kubectl rollout undo deployment/devops-app -n production
kubectl rollout status deployment/devops-app -n production
```

Expected: A previous revision becomes active.

Step 7 — Simulate bad image

Why: This reproduces a common deployment failure.

```
kubectl set image deployment/devops-app -n production devops-app=YOUR_DOCKERHUB_USERNAME/devops-app:does-not-exist
kubectl get pods -n production
kubectl rollout status deployment/devops-app -n production
```

Expected: `ErrImagePull` / `ImagePullBackOff` occurs.

Step 8 — Recover from bad image

Why: Rollback returns the Deployment to the known-good revision.

```
kubectl rollout undo deployment/devops-app -n production
kubectl rollout status deployment/devops-app -n production
kubectl get pods -n production
```

Expected: Production Pods become Ready again.

> ***Rollback vs Git revert:*** *Kubernetes rollback changes a Deployment revision. Git revert creates a new source-control change and normally triggers a new CI/CD build.*

15. Phase 11 — Security Hardening and Cleanup

Objective: Improve the security and operational quality of the project without blindly applying settings that could break the application.

Step 1 — Verify image UID

Why: The Kubernetes UID should match the image user before setting `runAsUser`.

```
docker run --rm YOUR_DOCKERHUB_USERNAME/devops-app:1.0.0 id
```

Expected: The image reports the `appuser` UID, commonly `1000` in this lab.

Step 2 — Add securityContext

Why: Non-root execution and disabling privilege escalation reduce container privileges.

```
securityContext:
  runAsNonRoot: true
  runAsUser: 1000
  allowPrivilegeEscalation: false
```

Step 3 — Use resource requests/limits

Why: Requests help scheduling; limits cap resource consumption.

```
resources:
  requests:
    cpu: 100m
    memory: 128Mi
  limits:
    cpu: 500m
    memory: 256Mi
```

Step 4 — Use rolling strategy

Why: `maxUnavailable=0` keeps existing capacity available while the new Pod is created.

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxUnavailable: 0
    maxSurge: 1
progressDeadlineSeconds: 120
```

Step 5 — Use labels

Why: Labels make resources easier to select and identify.

```
labels:
  app: devops-app
  project: devops-private-k8s-project
```

Step 6 — Add application health check to CI/CD

Why: A rollout can succeed while the application is still wrong; a `/health` check gives an application-level validation.

```
kubectl run qa-health-check --rm -i --restart=Never -n qa --image=curlimages/curl -- curl -f http://devops-app:5000/health
```

Expected: Command exits successfully.

Step 7 — Protect self-hosted runner

Why: Workflow code runs directly on the server.

Rules: - Keep the repository trusted/private. - Do not run arbitrary untrusted PR code on the runner. - Never expose runner tokens. - Prefer an isolated runner host in production.

Step 8 — Check Git safety

Why: This prevents credentials and infrastructure state from entering Git.

```
git status
git ls-files | grep -E '(\.env|tfstate|\.pem|\.key|terraform\.tfvars)'
git status --ignored
```

Expected: Sensitive files are not tracked.

Step 9 — Inspect disk before Docker cleanup

Why: The server contains unrelated workloads; broad pruning can break them.

```
df -h
docker system df
```

Expected: Review before cleanup. Do not blindly use `docker system prune -a`.

16. Phase 12 — Terraform + AWS S3 Remote State

Objective: Finish the infrastructure side. Terraform is deliberately separate from application deployment. In this project it manages AWS state infrastructure; K3s continues to manage the Flask application.

> ***Modern state design:*** *S3 stores* `terraform.tfstate` *and S3 native locking is enabled with* `use_lockfile=true`*. DynamoDB is not required for this design.*

Step 1 — Verify AWS identity

Why: Terraform needs AWS credentials with permission to create the state bucket.

```
aws sts get-caller-identity
```

Expected: AWS returns the active identity/account.

Step 2 — Create bootstrap directory

Why: The bootstrap configuration creates the S3 bucket before the main Terraform backend can use it.

```
mkdir -p ~/devops-private-k8s-project/terraform/bootstrap
cd ~/devops-private-k8s-project/terraform/bootstrap
```

Step 3 — Create bootstrap main.tf

Why: This creates an encrypted, versioned, private state bucket.

```
terraform {
  required_version = ">= 1.6.0"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}

resource "aws_s3_bucket" "terraform_state" {
  bucket = var.state_bucket_name
  tags = {
    Name        = "DevOps Terraform State"
    Project     = "devops-private-k8s-project"
    Environment = "shared"
  }
}

resource "aws_s3_bucket_versioning" "terraform_state" {
  bucket = aws_s3_bucket.terraform_state.id
  versioning_configuration {
    status = "Enabled"
  }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "terraform_state" {
  bucket = aws_s3_bucket.terraform_state.id
  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "AES256"
    }
  }
}

resource "aws_s3_bucket_public_access_block" "terraform_state" {
  bucket                  = aws_s3_bucket.terraform_state.id
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}
```

Step 4 — Create variables.tf

Why: The bucket name must be globally unique and the region should be configurable.

```
variable "aws_region" {
  description = "AWS region"
  type        = string
  default     = "ap-south-1"
}

variable "state_bucket_name" {
  description = "Globally unique Terraform state bucket"
  type        = string
}
```

Step 5 — Create terraform.tfvars

Why: This keeps local values out of main.tf; it must not be committed.

```
aws_region        = "ap-south-1"
state_bucket_name = "devops-private-k8s-tfstate-YOUR-UNIQUE-VALUE"
```

Step 6 — Initialize and validate

Why: Terraform syntax/provider/configuration errors should be caught before apply.

```
terraform init
terraform fmt
terraform validate
terraform plan
```

Expected: Validation succeeds and plan shows the bucket resources.

Step 7 — Apply bootstrap

Why: This creates the remote state bucket.

```
terraform apply
```

Expected: Bucket, versioning, encryption and public-access block are created.

Step 8 — Create main backend

Why: The root configuration now stores state remotely.

```
cd ~/devops-private-k8s-project/terraform
```

```
terraform {
  required_version = ">= 1.6.0"
  backend "s3" {
    bucket       = "YOUR_BUCKET_NAME"
    key          = "devops-private-k8s-project/terraform.tfstate"
    region       = "ap-south-1"
    use_lockfile = true
  }
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = "ap-south-1"
}
```

Step 9 — Initialize backend

Why: Terraform reads the backend and connects to S3.

```
terraform init
terraform plan
```

Expected: Initialization succeeds.

Step 10 — Migrate existing local state when applicable

Why: Migration safely moves existing local state into the configured backend.

```
terraform init -migrate-state
```

Expected: Terraform offers state migration when a local state exists.

Step 11 — Verify remote state

Why: This proves the state is no longer dependent on one workstation.

```
terraform state list
aws s3 ls s3://YOUR_BUCKET_NAME/devops-private-k8s-project/
```

Expected: The state object exists in S3.

> ***Credential best practice:*** *for GitHub Actions, prefer GitHub OIDC → AWS IAM role → temporary credentials rather than long-lived AWS keys.*

17. Final Repository Structure

```
devops-private-k8s-project/
├── app/
│   ├── app.py
│   ├── requirements.txt
│   ├── Dockerfile
│   └── .dockerignore
├── kubernetes/
│   ├── base/
│   │   ├── deployment.yaml
│   │   ├── service.yaml
│   │   └── kustomization.yaml
│   └── environments/
│       ├── qa/
│       │   ├── kustomization.yaml
│       │   └── ingress.yaml
│       ├── uat/
│       │   ├── kustomization.yaml
│       │   └── ingress.yaml
│       └── production/
│           ├── kustomization.yaml
│           └── ingress.yaml
├── ansible/
│   ├── ansible.cfg
│   ├── inventory/hosts
│   ├── group_vars/kubernetes.yml
│   ├── playbook.yml
│   └── roles/
│       ├── kubernetes/tasks/main.yml
│       └── github_runner/tasks/main.yml
├── .github/
│   └── workflows/
│       ├── ci.yml
│       └── terraform.yml
├── terraform/
│   ├── bootstrap/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── terraform.tfvars
│   └── main.tf
├── .gitignore
└── README.md
```

Recommended .gitignore

```
# Python
venv/
__pycache__/
*.pyc

# Terraform
.terraform/
*.tfstate
*.tfstate.*
crash.log
crash.*.log
terraform.tfvars

# Environment
.env
*.env
```

> *Do not commit `terraform.tfvars`, tfstate files, AWS credentials, Docker Hub tokens, GitHub runner tokens, `.pem`/`.key` files or other secrets.*

18. Final Verification Checklist

Area	Check	Expected
Server	`hostname`	`devops-lab`
K3s	`systemctl is-active k3s`	active
Node	`kubectl get nodes`	Ready
QA	`kubectl get pods -n qa`	2 Ready
UAT	`kubectl get pods -n uat`	2 Ready
Production	`kubectl get pods -n production`	3 Ready
Services	`kubectl get svc -A`	ClusterIP app services
Ingress	`kubectl get ingress -A`	3 routes
QA health	`curl http://qa.devops.local/health`	OK
UAT health	`curl http://uat.devops.local/health`	OK
Prod health	`curl http://prod.devops.local/health`	OK
Image promotion	compare Deployment images	same Git SHA
Runner	GitHub Settings → Runners	Online
CI	GitHub Actions	test/build/push
UAT gate	GitHub Environment	approval required
Prod gate	GitHub Environment	approval required
Self-healing	delete a Pod	replacement appears
Scaling	`kubectl scale`	replica count changes
Rollback	`kubectl rollout undo`	previous revision
Terraform	`terraform validate`	success
Remote state	`aws s3 ls ...`	state exists
Git safety	`git ls-files` check	no secrets/state

19. Troubleshooting

K3s not running

```
systemctl status k3s
journalctl -u k3s -n 100 --no-pager
kubectl get nodes
```

Pod is ImagePullBackOff

```
kubectl get pods -n production
kubectl describe pod <pod-name> -n production
```

Check the exact Docker Hub image/tag. If the repository is private, configure `imagePullSecrets`.

Deployment rollout failed

```
kubectl rollout status deployment/devops-app -n production
kubectl describe deployment devops-app -n production
kubectl get pods -n production
kubectl describe pod <pod-name> -n production
kubectl logs <pod-name> -n production
```

Ingress does not route

```
kubectl get ingress -A
kubectl describe ingress devops-app -n qa
kubectl get svc -n qa
kubectl get endpoints -n qa
kubectl get pods -n qa
curl -v -H "Host: qa.devops.local" http://172.16.4.110/health
```

Runner offline

```
systemctl --type=service | grep actions.runner
sudo ./svc.sh status
journalctl -u <runner-service> -n 100 --no-pager
```

Actions job cannot run kubectl

```
which kubectl
kubectl version --client
kubectl get nodes
whoami
id
```

The workflow runs as the runner user. That user needs the correct kubectl/K3s permissions and access to required files.

Terraform backend error

```
aws sts get-caller-identity
terraform init
terraform validate
terraform plan
```

If a local state file exists and needs migration, use `terraform init -migrate-state` rather than deleting it.

Disk pressure

```
df -h
docker system df
kubectl describe node devops-lab | grep -i -A5 pressure
```

> *Never blindly run `docker system prune -a` on this server because unrelated workloads may depend on local images.*

20. Interview Guide and Definitions

Project explanation — memorize this flow:

> *"I built a private Kubernetes-based DevOps project for a Flask application. GitHub stores the source code. GitHub Actions tests the application, builds one Docker image and pushes it to Docker Hub using the Git commit SHA. The same immutable image is promoted through QA, UAT and Production on a private K3s cluster. Kustomize manages environment-specific configuration, Traefik provides ingress routing, UAT and Production have manual approval gates, and Kubernetes readiness/liveness probes, rolling updates, scaling and rollback are included. Ansible configures the server and self-hosted runner, while Terraform is kept separate for infrastructure and AWS S3 remote state."*

Why Ansible + Terraform? Terraform provisions/manages infrastructure. Ansible configures servers. In this project Terraform is not used to deploy the Flask application into Kubernetes.

Why Docker + Kubernetes? Docker packages the application. Kubernetes orchestrates the containers and provides scaling, self-healing, service discovery and rollout management.

Why Kustomize? It lets us keep one common base and apply small environment-specific changes rather than copying full YAML files.

Why Traefik? It is the HTTP entry point. It receives requests and routes them to the correct Service based on hostname.

Why Git SHA tags? They make images immutable and traceable to the exact source commit. This is safer than relying on a mutable `latest` tag.

Key terms

Term	Simple interview definition
CI	Automatically build and test source changes.
CD	Automate delivery/deployment of validated changes.
Pod	Smallest deployable Kubernetes unit.
Deployment	Maintains desired Pod replicas and manages rolling updates.
Service	Stable network endpoint for Pods.
ClusterIP	Internal Service type used behind Traefik.
Ingress	Defines HTTP/HTTPS routing rules.
Traefik	Ingress controller that implements those routing rules.
Kustomize	Builds environment-specific manifests from a common base.
Readiness probe	Determines whether a Pod can receive traffic.
Liveness probe	Determines whether a container is healthy enough to continue.
Desired state	The configuration Kubernetes is expected to maintain.
Self-healing	Kubernetes corrects differences between actual and desired state, such as recreating a deleted Pod.
Rolling update	Gradually replaces old Pods with new Pods.
Rollback	Returns a Deployment to an earlier revision.
Ansible	Server configuration and automation tool.
Terraform	Infrastructure as Code tool.
Terraform state	Record of resources Terraform manages.
S3 backend	Remote Terraform state storage; this project also enables S3 native locking.
Self-hosted runner	A machine managed by you that executes GitHub Actions jobs.

21. Final Mental Model

```
CODE
 |
 v
GITHUB
 |
 v
GITHUB ACTIONS
 |---- TEST
 |---- DOCKER BUILD
 `---- PUSH IMAGE
 |
 v
DOCKER HUB
 |
 v
 QA
 |
APPROVAL
 |
 v
 UAT
 |
APPROVAL
 |
 v
PRODUCTION
 |
 v
K3s / Kubernetes
 |
Traefik
 |
User-facing hostnames

Ansible   -> server + runner configuration
Terraform -> AWS infrastructure/state
S3        -> remote Terraform state
```

Core principle: Build once, test once, and promote the same immutable artifact through controlled environments.

This guide is intended to be sufficient for rebuilding the lab from scratch: follow the phases in order, replace placeholders with your own values, verify each phase before moving forward, and never expose secrets.

End of complete step-by-step guide.